```
> # ================================================================
==
# TauTimp — Cylindrical Symmetry: Variable-Coefficient Wave
Equation
# f(r,z,t) = H3(r) * H4(z) * u(t),  BC: H3(R)=0, H4 sinusoidal
# Photon probability density P(r,z,t): d'Alembert, BC P(R,z,0)=0
# Velocities: V=c, V=10^9*c
# Author: Viorel Popescu
# Maple Transcript — Cylindrical Framework v1.0
# ================================================================
==

restart;
with(plots):
with(plottools):
with(StringTools):
Digits := 15:

# ================================================================
==
# SECTION 1: PHYSICAL PARAMETERS
# ================================================================
==

R  := 4.235518529e22:              # [m]  TauTimp cylindrical
radius
c  := 2.99792458e8:           # [m/s] speed of light
V1 := c:                      # photon speed
V2 := 1.0e9 * c:              # onduscular speed

# Axial domain (choose L = 2*R for comparability)
L_z := 2.0 * R:               # [m]  axial domain length

printf("=== TauTimp Cylindrical Parameters ===\n"):
printf("R    = %.6e m\n", R):
printf("L_z  = %.6e m\n", L_z):
printf("c    = %.8e m/s\n", c):
printf("V1   = %.6e m/s  (light)\n", V1):
printf("V2   = %.6e m/s  (onduscular)\n", V2):

# ================================================================
==
```

```
# SECTION 2: AIRY ZEROS — a_n : Ai(-a_n) = 0
# ================================================================
==

N_modes := 6:

airy_zero := proc(n::posint)
    local guess;
    guess := evalf((3.0*Pi*(4*n-1)/8)^(2.0/3.0));
    fsolve(AiryAi(-x) = 0, x = guess - 0.5 .. guess + 0.5);
end proc:

a := Array(1..N_modes):
printf("\n=== Airy Zeros a_n  [Ai(-a_n)=0] ===\n"):
for n from 1 to N_modes do
    a[n] := airy_zero(n);
    printf("a[%d] = %.8f\n", n, evalf(a[n]));
end do:

# ================================================================
==
# SECTION 3: RADIAL EIGENVALUES k_n (k_z=0 case)
# ================================================================
==
# omega_n = a_n^(3/2) / R  (in units where c=1)
# Physical freq: Omega_n = omega_n * c = c * a_n^(3/2) / R

omega_r := n -> evalf(a[n]^(3/2) / R):          # [rad/m]
dimensionless
Omega_n  := n -> evalf(a[n]^(3/2) * c / R):      # [rad/s]
physical

printf("\n=== Radial Eigenfrequencies (k_z=0, V=c) ===\n"):
printf("%-5s  %-22s  %-22s\n", "n", "k_n [m^-1]", "Omega_n
[rad/s]"):
for n from 1 to N_modes do
    printf("%-5d  %-22.6e  %-22.6e\n", n, omega_r(n), Omega_n(n))
;
end do:

# ================================================================
==
```

```
# SECTION 4: AIRY VARIABLE AND EIGENFUNCTIONS  (k_z=0)
# ================================================================
==
# zeta_n(r) = (omega_n^2/R)^(1/3) * r  = a_n * r / R  (at k_z=0)

zeta_n := (r_val, n) -> evalf(a[n] * r_val / R):

# Radial eigenfunction H3^(n)(r) = Ai(-zeta_n(r)) / sqrt(r)
H3 := (r_val, n) -> evalf(AiryAi(-zeta_n(r_val, n)) / sqrt(r_val)
):

# BC verification
printf("\n=== BC Verification: H3(R, n) should be ~0 ===\n"):
for n from 1 to 5 do
    printf("H3(R, %d) = %.3e\n", n, H3(R, n));
end do:

# ================================================================
==
# SECTION 5: AXIAL FACTOR H4(z) — sinusoidal
# ================================================================
==
# H4_m(z) = cos(k_m * z)  or  sin(k_m * z),   k_m = m*Pi/L_z

H4c := (z_val, m) -> cos(m * Pi * z_val / L_z):
H4s := (z_val, m) -> sin(m * Pi * z_val / L_z):

# ================================================================
==
# SECTION 6: TEMPORAL FACTOR u(t)
# ================================================================
==
# Omega_nm^2 = (a_n^(3/2)/R * c)^2 + (m*Pi*c/L_z)^2 (dispersion)
# For k_z=0: Omega_n = a_n^(3/2)*c/R

Omega_nm := (n_r, m_z) -> evalf(
    sqrt((a[n_r]^(3/2) * c / R)^2 + (m_z * Pi * c / L_z)^2)):

u_cos := (t_val, n_r, m_z) -> cos(Omega_nm(n_r, m_z) * t_val):
u_sin := (t_val, n_r, m_z) -> sin(Omega_nm(n_r, m_z) * t_val):

printf("\n=== Eigenfrequencies Omega_nm [rad/s] ===\n"):
```

```
printf("%-6s  %-6s  %-22s\n", "n_r", "m_z", "Omega_nm"):
for n_r from 1 to 3 do
    for m_z from 0 to 2 do
        printf("%-6d  %-6d  %-22.6e\n", n_r, m_z, Omega_nm(n_r,
m_z));
    end do:
end do:

# Full mode: f_nm(r, z, t) at t=0
f_mode := (r_val, z_val, t_val, n_r, m_z) ->
    H3(r_val, n_r) * H4c(z_val, m_z) * u_cos(t_val, n_r, m_z):

# ================================================================
==
# SECTION 7: PLOT 1 — Radial eigenfunctions H3(r,n), n=1..4
# ================================================================
==

Nr := 400:
r_vec := [seq(0.001*R + (R - 0.001*R)*(i-1)/(Nr-1), i=1..Nr)]:
col4 := [blue, red, "DarkGreen", purple]:

curves_H3 := []:
for n from 1 to 4 do
    dat := [seq([r_vec[i]/R, H3(r_vec[i], n)], i=1..Nr)]:
    curves_H3 := [op(curves_H3),
        listplot(dat, color=col4[n], thickness=2, linestyle=
solid,
            legend=cat("n=", n))]:
end do:

p1 := display(curves_H3,
    title  = "Cylindrical Airy Eigenfunctions H3(r,n)  [H3(R)=0]
",
    labels = ["r / R", "H3(r,n)  [a.u.]"],
    labeldirections = [horizontal, vertical],
    axes   = boxed, gridlines = true, size = [900, 440]):
print(p1):

# ================================================================
==
# SECTION 8: PLOT 2 — Axial factor H4(z,m), m=1..3
```

```
# ================================================================
==

Nz := 300:
z_vec := [seq((i-1)*L_z/(Nz-1), i=1..Nz)]:

curves_H4 := []:
for m from 1 to 3 do
    dat := [seq([z_vec[i]/R, H4c(z_vec[i], m)], i=1..Nz)]:
    curves_H4 := [op(curves_H4),
        listplot(dat, color=col4[m], thickness=2, linestyle=
solid,
            legend=cat("m=",m))]:
end do:

p2 := display(curves_H4,
    title  = "Axial Factor H4(z,m) = cos(m*Pi*z/L_z) —
Sinusoidal Modes",
    labels = ["z / R", "H4(z,m)"],
    axes   = boxed, gridlines = true, size = [900, 380]):
print(p2):

# ================================================================
==
# SECTION 9: PLOT 3 — 2D slice f(r,z,0): n=1, m=1
# ================================================================
==

Nr2d := 60: Nz2d := 60:
r_2d := [seq(0.001*R + (R-0.001*R)*(i-1)/(Nr2d-1), i=1..Nr2d)]:
z_2d := [seq((j-1)*L_z/(Nz2d-1), j=1..Nz2d)]:

data_f2d := []:
for i from 1 to Nr2d do
    for j from 1 to Nz2d do
        fval := evalf(f_mode(r_2d[i], z_2d[j], 0, 1, 1));
        data_f2d := [op(data_f2d), [r_2d[i]/R, z_2d[j]/R, fval]]:
    end do:
end do:

p3 := pointplot3d(data_f2d,
    symbol=point, symbolsize=3, color=z,
```

```
    title  = "f(r,z,0) = H3(r,1)*H4(z,1)*1 — Cylindrical mode n=
1,m=1",
    labels = ["r/R", "z/R", "f"],
    axes   = boxed, orientation=[50,55], size=[900,550]):
print(p3):

# ================================================================
==
# SECTION 10: PLOT 4 — Time snapshots f(r,z,t) at z=L_z/2, n=1,
m=1
# ================================================================
==

T_11 := evalf(2*Pi / Omega_nm(1,1)):
printf("\nOmega(n=1,m=1) = %.6e rad/s,  T = %.6e s\n",
    Omega_nm(1,1), T_11):

t_snaps4 := [0, T_11/4, T_11/2, 3*T_11/4]:
lab4 := ["t=0", "t=T/4", "t=T/2", "t=3T/4"]:
z_fixed := L_z / 2:

curves_ft := []:
for j from 1 to 4 do
    dat := [seq([r_vec[i]/R,
        f_mode(r_vec[i], z_fixed, t_snaps4[j], 1, 1)], i=1..Nr)]:
    curves_ft := [op(curves_ft),
        listplot(dat, color=col4[j], thickness=2, linestyle=
solid,
            legend=lab4[j])]:
end do:

p4 := display(curves_ft,
    title  = "f(r, z=L/2, t): Cylindrical n=1,m=1 — Time
Snapshots",
    labels = ["r / R", "f(r,L/2,t)  [a.u.]"],
    axes   = boxed, gridlines = true, size=[900,420]):
print(p4):

# ================================================================
==
# SECTION 11: PLOT 5 — Mode superposition f(r,z,0): n=1,2; m=1,2
# ================================================================
```

```
==

f_super := (r_val, z_val, t_val) ->
    evalf(f_mode(r_val,z_val,t_val,1,1)
        + 0.5*f_mode(r_val,z_val,t_val,2,1)
        + 0.3*f_mode(r_val,z_val,t_val,1,2)):

data_sup := []:
for i from 1 to Nr2d do
    for j from 1 to Nz2d do
        data_sup := [op(data_sup),
            [r_2d[i]/R, z_2d[j]/R, f_super(r_2d[i], z_2d[j], 0)]]
:
    end do:
end do:

p5 := pointplot3d(data_sup,
    symbol=point, symbolsize=3, color=z,
    title  = "Superposition f(r,z,0): n=1,2; m=1,2 — Cylindrical
modes",
    labels = ["r/R", "z/R", "f"],
    axes   = boxed, orientation=[45,60], size=[900,550]):
print(p5):

# ============================================================
==
# SECTION 12: d'ALEMBERT PROBABILITY DENSITY P(r,z,t)
# ============================================================
==
# d'Alembert in r: d^2P/dt^2 - V^2 * d^2P/dr^2 = 0
# BC: P(R, z, 0) = 0  =>  P_n(r,z,t) = A(z)*sin(n*Pi*r/R)*cos(n*
Pi*V*t/R)

P_mode := proc(r_val, z_val, t_val, n_mode, m_z, V_val)
    local omega_P, k_z;
    omega_P := n_mode * Pi * V_val / R;
    k_z     := m_z * Pi / L_z;
    sin(n_mode * Pi * r_val / R) *
    (cos(k_z * z_val) + sin(k_z * z_val)) *
    cos(omega_P * t_val);
end proc:
```

```
# BC check
printf("\n=== d'Alembert BC: P(R,z,0,n,m,V) should be 0 ===\n"):
for n from 1 to 3 do
    printf("P_dA(R, 0, 0, %d, 1, V1) = %.3e\n", n,
        evalf(P_mode(R, 0.5*L_z, 0, n, 1, V1)));
end do:

# Frequencies
printf("\n=== d'Alembert Frequencies omega_n = n*Pi*V/R ===\n"):
printf("%-5s  %-22s  %-22s\n", "n", "V=c [rad/s]", "V=10^9c
[rad/s]"):
for n from 1 to 5 do
    printf("%-5d  %-22.6e  %-22.6e\n", n,
        n*Pi*V1/R, n*Pi*V2/R);
end do:

# ================================================================
==
# SECTION 13: PLOT 6 — P(r,z,0) spatial map, V=c, n=1, m=1
# ================================================================
==

data_P2d := []:
for i from 1 to Nr2d do
    for j from 1 to Nz2d do
        Pval := evalf(P_mode(r_2d[i], z_2d[j], 0, 1, 1, V1));
        data_P2d := [op(data_P2d), [r_2d[i]/R, z_2d[j]/R, Pval]]:
    end do:
end do:

p6 := pointplot3d(data_P2d,
    symbol=point, symbolsize=3, color=z,
    title  = "P(r,z,0) — d'Alembert n=1,m=1, V=c  [P(R,z,0)=0]",
    labels = ["r/R", "z/R", "P"],
    axes   = boxed, orientation=[45,60], size=[900,550]):
print(p6):

# ================================================================
==
# SECTION 14: PLOT 7 — P(r,z,0) at fixed z=L/2, modes n=1..4, V=
c
# ================================================================
```

```
==

z_fix2 := L_z / 2:
curves_Pn := []:
for n from 1 to 4 do
    dat := [seq([r_vec[i]/R,
        P_mode(r_vec[i], z_fix2, 0, n, 1, V1)], i=1..Nr)]:
    curves_Pn := [op(curves_Pn),
        listplot(dat, color=col4[n], thickness=2, linestyle=
solid,
            legend=cat("n=",n))]:
end do:

p7 := display(curves_Pn,
    title  = "P(r, z=L/2, 0) — d'Alembert Modes n=1..4, V=c",
    labels = ["r / R", "P(r,L/2,0)  [a.u.]"],
    axes   = boxed, gridlines = true, size=[900,420]):
print(p7):

# ============================================================
==
# SECTION 15: PLOT 8 — Velocity comparison P(r, L/2, 0): V=c vs
V=10^9c
# ============================================================
==
# At t=0 the spatial profile is IDENTICAL for both velocities
# Overlay n=1 profiles and annotate frequency ratio

dat_Vc  := [seq([r_vec[i]/R, P_mode(r_vec[i], z_fix2, 0, 1, 1,
V1)], i=1..Nr)]:
dat_Vo  := [seq([r_vec[i]/R, P_mode(r_vec[i], z_fix2, 0, 1, 1,
V2)], i=1..Nr)]:

p8 := display(
    listplot(dat_Vc, color=blue, thickness=3, legend="V=c",
        linestyle=solid),
    listplot(dat_Vo, color=red,  thickness=2, legend="V=10^9*c
(same profile)",
        linestyle=dash),
    title  = "P(r, L/2, 0): V=c vs V=10^9c — profiles identical
at t=0",
    labels = ["r / R", "P  [a.u.]"],
```

```
    axes   = boxed, gridlines = true, size=[900,400]):
print(p8):

# ================================================================
==
# SECTION 16: PLOT 9 — Time evolution P(r,L/2,t) at r=R/2
# ================================================================
==

r_half := R / 2:
T_P1_c := evalf(2*R / V1):
T_P1_o := evalf(2*R / V2):
Npt    := 600:

printf("\n=== d'Alembert Periods ===\n"):
printf("V=c      : T_1 = %.4e s\n", T_P1_c):
printf("V=10^9*c : T_1 = %.4e s\n", T_P1_o):

# V=c: plot 3 periods
t_c := [seq(3*T_P1_c*(i-1)/(Npt-1), i=1..Npt)]:
dat_tc := [seq([t_c[i]/T_P1_c,
    P_mode(r_half, z_fix2, t_c[i], 1, 1, V1)], i=1..Npt)]:

p9a := listplot(dat_tc, color=blue, thickness=2,
    title  = "P(R/2, L/2, t) — V=c, n=1  [t in units of T_1]",
    labels = ["t / T_1", "P"],
    axes   = boxed, gridlines=true, size=[900,300]):
print(p9a):

# V=10^9*c: plot 3 periods
t_o := [seq(3*T_P1_o*(i-1)/(Npt-1), i=1..Npt)]:
dat_to := [seq([t_o[i]/T_P1_o,
    P_mode(r_half, z_fix2, t_o[i], 1, 1, V2)], i=1..Npt)]:

p9b := listplot(dat_to, color=red, thickness=2,
    title  = "P(R/2, L/2, t) — V=10^9c, n=1  [t in units of T_1]
",
    labels = ["t / T_1", "P"],
    axes   = boxed, gridlines=true, size=[900,300]):
print(p9b):

# ================================================================
```

```
==
# SECTION 17: PLOT 10 — 2D time-space map P(r,t) at z=L/2, V=c
# ================================================================
==

Nr3 := 80: Nt3 := 80:
r_grid3 := [seq(0.001*R + (R-0.001*R)*(i-1)/(Nr3-1), i=1..Nr3)]:
t_grid3 := [seq(T_P1_c*(j-1)/(Nt3-1), j=1..Nt3)]:

data_Prt := []:
for i from 1 to Nr3 do
    for j from 1 to Nt3 do
        Pval := evalf(P_mode(r_grid3[i], z_fix2, t_grid3[j], 1,
1, V1));
        data_Prt := [op(data_Prt),
            [r_grid3[i]/R, t_grid3[j]/T_P1_c, Pval]]:
    end do:
end do:

p10 := pointplot3d(data_Prt,
    symbol=point, symbolsize=2, color=z,
    title  = "P(r, z=L/2, t) — d'Alembert, V=c  [axes: r/R, t/T,
P]",
    labels = ["r/R", "t/T", "P"],
    axes   = boxed, orientation=[50,65], size=[900,550]):
print(p10):

# ================================================================
==
# SECTION 18: PLOT 11 — WKB vs Exact Airy comparison
(cylindrical)
# ================================================================
==

Ai_WKB := proc(zeta_val)
    if zeta_val < 0.5 then return evalf(AiryAi(-zeta_val)) end
if;
    (1.0/(sqrt(Pi)*zeta_val^(1.0/4.0)))*
        sin(2.0/3.0*zeta_val^(3.0/2.0)+Pi/4.0);
end proc:

dat_exact := [seq([r_vec[i]/R,
```

```
    evalf(AiryAi(-zeta_n(r_vec[i],1))/sqrt(r_vec[i]))], i=1..Nr)]
:
dat_wkb   := [seq([r_vec[i]/R,
    Ai_WKB(zeta_n(r_vec[i],1))/sqrt(r_vec[i])], i=1..Nr)]:

p11 := display(
    listplot(dat_exact, color=blue,  thickness=3, legend="Exact
Ai(-ζ)"),
    listplot(dat_wkb,   color=red,   thickness=2, legend="WKB",
linestyle=dash),
    title  = "Cylindrical H3(r,n=1): Exact Airy vs WKB",
    labels = ["r / R", "H3(r,1)  [a.u.]"],
    axes   = boxed, gridlines=true, size=[900,420]):
print(p11):

# ============================================================
==
# SECTION 19: PLOT 12 — Dispersion relation Omega(k_z) for n=1,
2,3
# ============================================================
==

N_kz := 100:
kz_max := 5*Pi/R:
kz_vec := [seq((i-1)*kz_max/(N_kz-1), i=1..N_kz)]:

curves_disp := []:
for n_r from 1 to 3 do
    dat_disp := [seq([kz_vec[i]*R/Pi,
        sqrt((a[n_r]^(3/2)*c/R)^2 + (kz_vec[i]*c)^2)], i=1..N_kz)
]:
    curves_disp := [op(curves_disp),
        listplot(dat_disp, color=col4[n_r], thickness=2,
            legend=cat("n=",n_r))]:
end do:

p12 := display(curves_disp,
    title  = "Dispersion Relation Omega(k_z) for n=1,2,3 —
Cylindrical TauTimp",
    labels = ["k_z * R / Pi", "Omega [rad/s]"],
    axes   = boxed, gridlines=true, size=[900,420]):
print(p12):
```

```
# ================================================================
==
# SECTION 20: PLOT 13 — Normalization integrals I_n
# ================================================================
==

printf("\n=== Normalization Integrals I_n = integral_0^{a_n} [Ai
(-rho)]^2 drho ===\n"):
I_vals := Array(1..4):
for n from 1 to 4 do
    I_vals[n] := evalf(Int(AiryAi(-rho)^2, rho=0..a[n],
        method=_d01ajc, epsilon=1e-8));
    N_n := I_vals[n] * R^(1/3) / (a[n]^(3/2)/R)^(2/3):
    printf("n=%d: I_%d=%.6f,  N_%d=%.4e m\n",
        n, n, evalf(I_vals[n]), n, evalf(N_n));
end do:

# Bar chart of I_n
dat_bar := [seq([n, evalf(I_vals[n])], n=1..4)]:
p13 := listplot(dat_bar,
    style=point, symbol=solidcircle, symbolsize=16,
    color=blue, thickness=2,
    title  = "Normalization integrals I_n vs mode index n",
    labels = ["n", "I_n = integral_0^{a_n} [Ai(-rho)]^2 d_rho"],
    axes   = boxed, gridlines=true, size=[700,380]):
print(p13):

# ================================================================
==
# SECTION 21: PLOT 14 — Angular extension: H(r,theta,z,t) at t=0
# ================================================================
==
# Add azimuthal modulation e^{i*m_theta*theta} -> cos(m_theta*
theta)
# H = f(r,z,0) * [1 + 0.4*cos(theta) + 0.2*cos(2*theta)]
# Convert to Cartesian for display: x=r*cos(theta), y=r*sin
(theta)

N_ang := 36: Nr_cyl := 30: Nz_cyl := 20:
r_cyl := [seq(0.01*R + (R-0.01*R)*(i-1)/(Nr_cyl-1), i=1..Nr_cyl)]
:
z_cyl := [seq((j-1)*L_z/(Nz_cyl-1), j=1..Nz_cyl)]:
```

```
th_cyl := [seq(2*Pi*(k-1)/(N_ang-1), k=1..N_ang)]:

ang_mod := theta_val -> (1.0 + 0.4*cos(theta_val) + 0.2*cos(2*
theta_val)):

data_Hcyl := []:
for i from 1 to Nr_cyl do
    for j from 1 to Nz_cyl do
        f_rz := evalf(H3(r_cyl[i],1) * H4c(z_cyl[j],1));
        for k from 1 to N_ang do
            th := th_cyl[k];
            Hval := f_rz * ang_mod(th);
            x_c  := r_cyl[i] * cos(th) / R;
            y_c  := r_cyl[i] * sin(th) / R;
            z_c  := z_cyl[j] / R;
            data_Hcyl := [op(data_Hcyl), [x_c, y_c, z_c, Hval]]:
        end do:
    end do:
end do:

# Extract (x,y,z) and color by Hval
pts_xyz := [seq([data_Hcyl[i][1], data_Hcyl[i][2], data_Hcyl[i]
[3]],
    i=1..nops(data_Hcyl))]:

p14 := pointplot3d(pts_xyz,
    symbol=point, symbolsize=2,
    title  = "H(r,theta,z,0): Cylindrical field with azimuthal
modulation",
    labels = ["x/R", "y/R", "z/R"],
    axes   = boxed, orientation=[30,50], size=[900,600]):
print(p14):

# ================================================================
==
# SECTION 22: SUMMARY TABLE
# ================================================================
==

printf("\n"):
printf("%s\n", Repeat("=", 92)):
printf("  GENERALIZED CYLINDRICAL FRAMEWORK SUMMARY ——
```

```
  TauTimp\n"):
  printf("%s\n", Repeat("=", 92)):
  printf("  R = %.4e m,   L_z = %.4e m,   c = %.6e m/s\n", R, L_z,
  c):
  printf("%s\n", Repeat("-", 92)):
  printf("  %-6s  %-6s  %-12s  %-10s  %-22s  %-16s\n",
      "n_r","m_z","Method","V [m/s]","Omega [rad/s]","T [s]"):
  printf("%s\n", Repeat("-", 92)):

  rows_cyl := [
      [1,0,"Airy",    "c",      Omega_nm(1,0), 2*Pi/Omega_nm(1,0)],
      [2,0,"Airy",    "c",      Omega_nm(2,0), 2*Pi/Omega_nm(2,0)],
      [1,1,"Airy",    "c",      Omega_nm(1,1), 2*Pi/Omega_nm(1,1)],
      [2,1,"Airy",    "c",      Omega_nm(2,1), 2*Pi/Omega_nm(2,1)],
      [1,0,"d'Alem.", "c",      Pi*V1/R,       2*R/V1],
      [1,0,"d'Alem.", "10^9*c", Pi*V2/R,       2*R/V2],
      [2,0,"d'Alem.", "c",      2*Pi*V1/R,     R/V1],
      [2,0,"d'Alem.", "10^9*c", 2*Pi*V2/R,     R/V2]
  ]:

  for row in rows_cyl do
      printf("  %-6d  %-6d  %-12s  %-10s  %-22.6e  %-16.4e\n",
          row[1], row[2], row[3], row[4],
          evalf(row[5]), evalf(row[6]));
  end do:
  printf("%s\n", Repeat("=", 92)):

  # ================================================================
  ==
  # DONE
  # ================================================================
  ==

  printf("\n=== TauTimp Cylindrical Transcript Complete ===\n"):
  printf("Plots generated: p1..p14 (14 figures)\n"):
  printf("Files: TauTimp_CylindricalSymmetry.md +
  TauTimp_CylindricalSymmetry.mpl\n"):
```

```
=== TauTimp Cylindrical Parameters ===
R    = 4.235519e+22 m
L_z  = 8.471037e+22 m
c    = 2.99792458e+08 m/s
V1   = 2.997925e+08 m/s  (light)
```

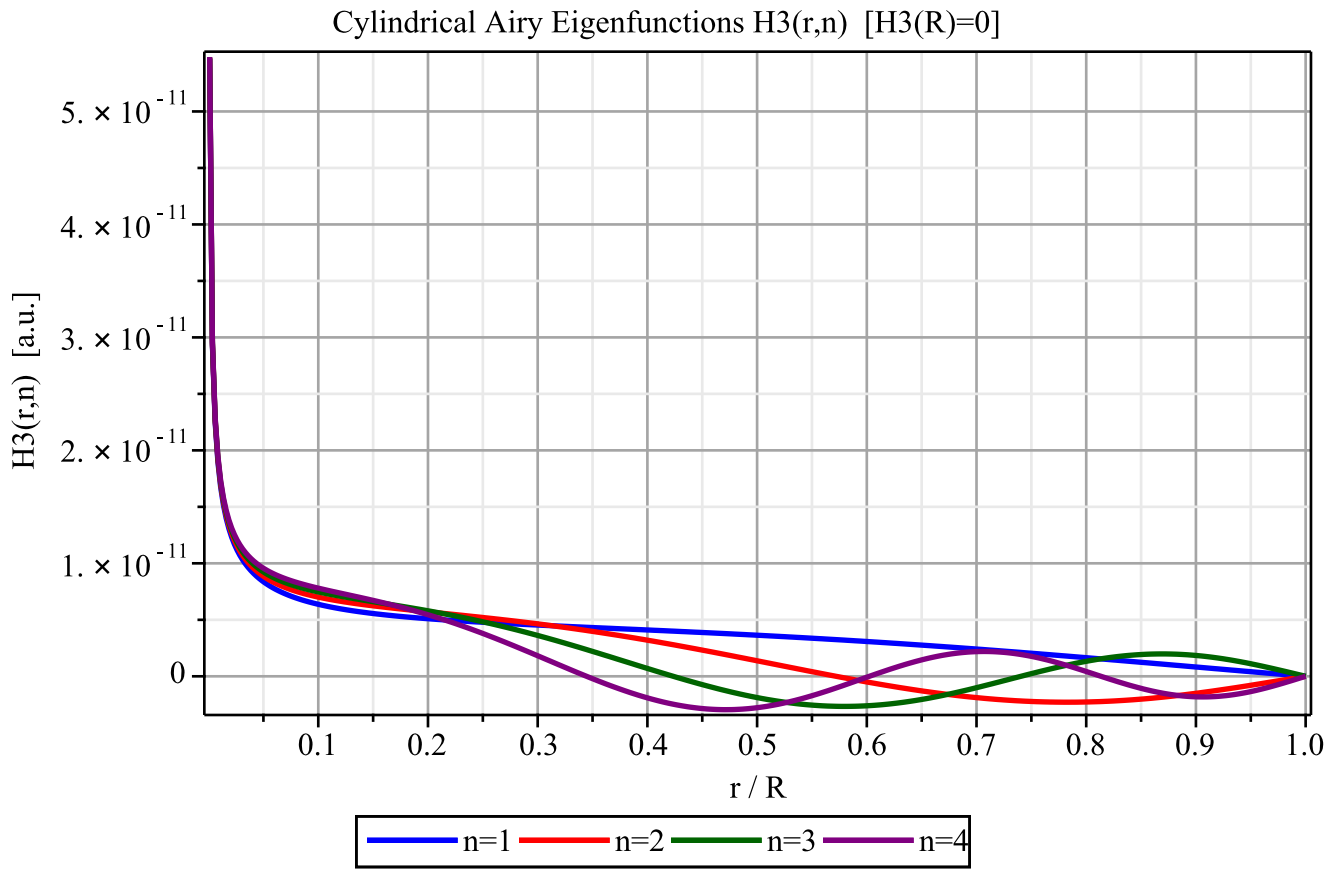
```
V2   = 2.997925e+17 m/s  (onduscular)

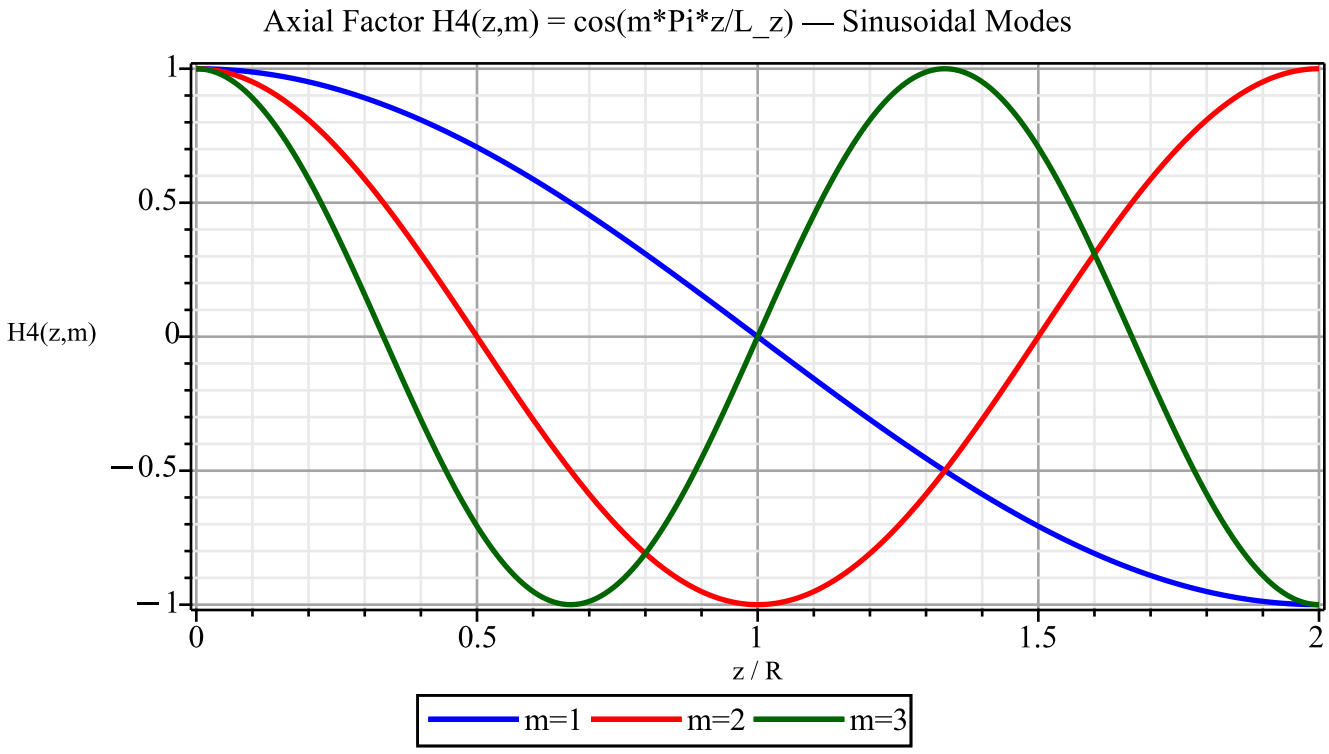
=== Airy Zeros a_n  [Ai(-a_n)=0] ===
a[1] = 2.33810741
a[2] = 4.08794944
a[3] = 5.52055983
a[4] = 6.78670809
a[5] = 7.94413359
a[6] = 9.02265085

=== Radial Eigenfrequencies (k_z=0, V=c) ===
n       k_n [m^-1]               Omega_n [rad/s]
1       8.440926e-23             2.530526e-14
2       1.951424e-22             5.850223e-14
3       3.062444e-22             9.180975e-14
4       4.174286e-22             1.251419e-13
5       5.286439e-22             1.584835e-13
6       6.398743e-22             1.918295e-13

=== BC Verification: H3(R, n) should be ~0 ===
H3(R, 1) = -1.009e-26
H3(R, 2) = 3.662e-26
H3(R, 3) = -3.759e-26
H3(R, 4) = -3.983e-26
H3(R, 5) = 1.438e-26

=== Eigenfrequencies Omega_nm [rad/s] ===
n_r     m_z     Omega_nm
1       0       2.530526e-14
1       1       2.764001e-14
1       2       3.368698e-14
2       0       5.850223e-14
2       1       5.954935e-14
2       2       6.258568e-14
3       0       9.180975e-14
3       1       9.248051e-14
3       2       9.446421e-14
```

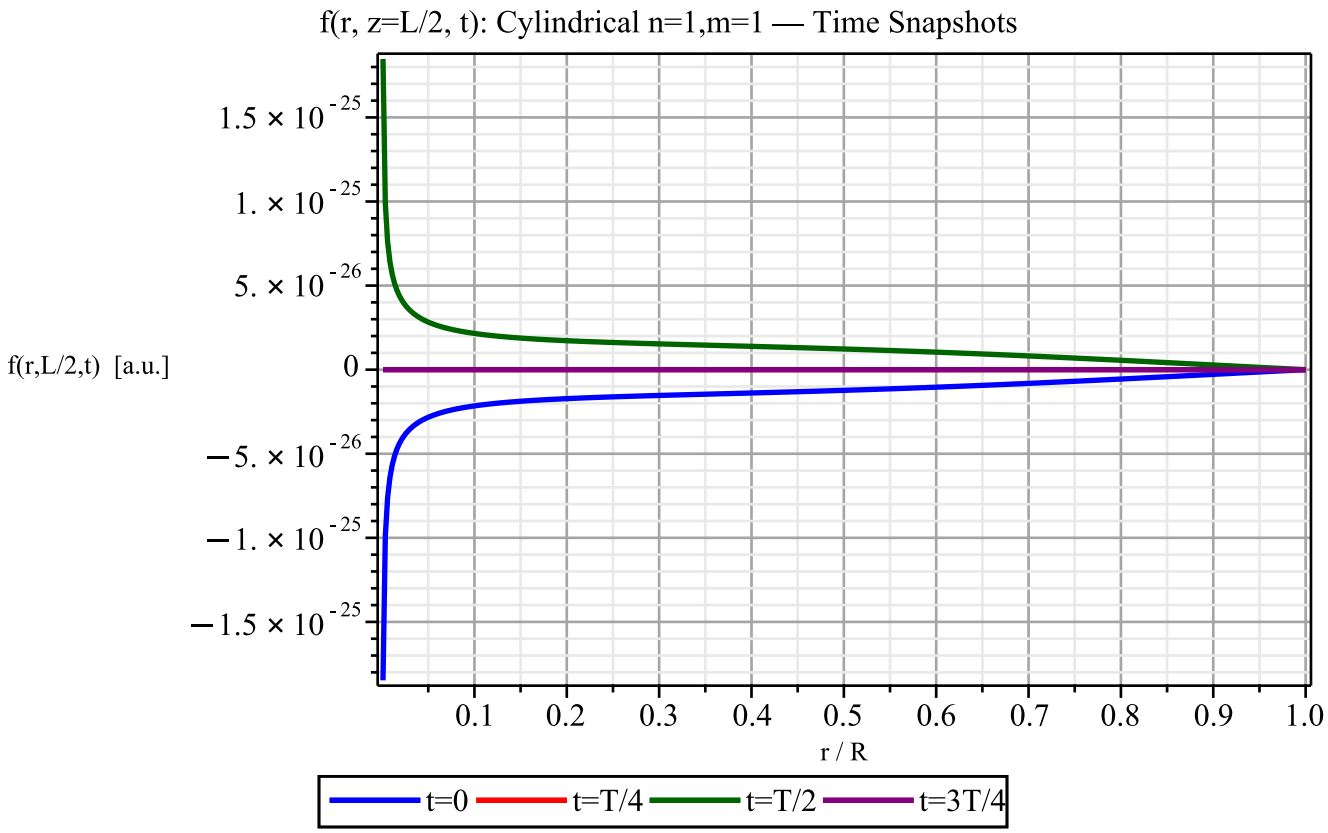




Error, (in plots:-pointplot3d) invalid color specification: z

p3

```
Omega(n=1,m=1) = 2.764001e-14 rad/s,  T = 2.273221e+14 s
```



Error, (in plots:-pointplot3d) invalid color specification: z

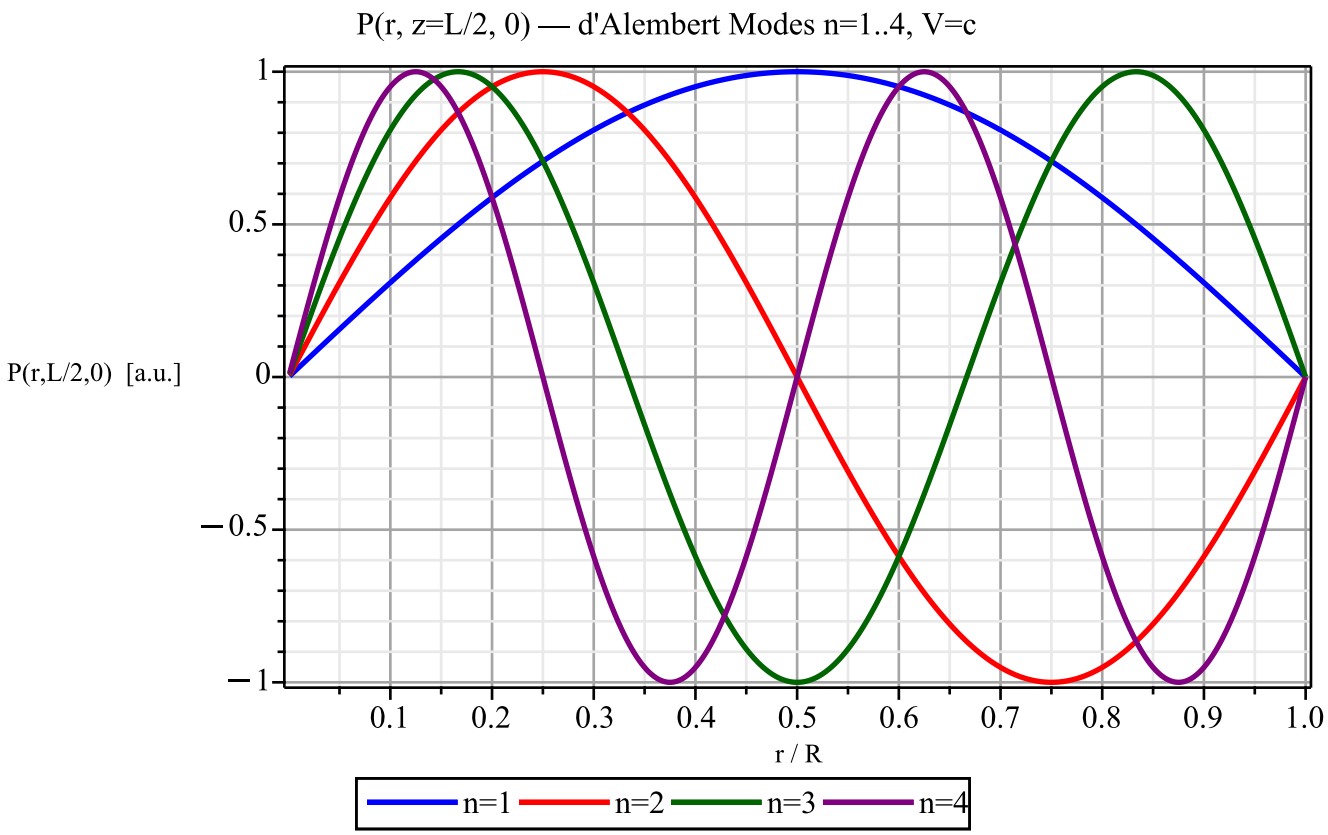
p5

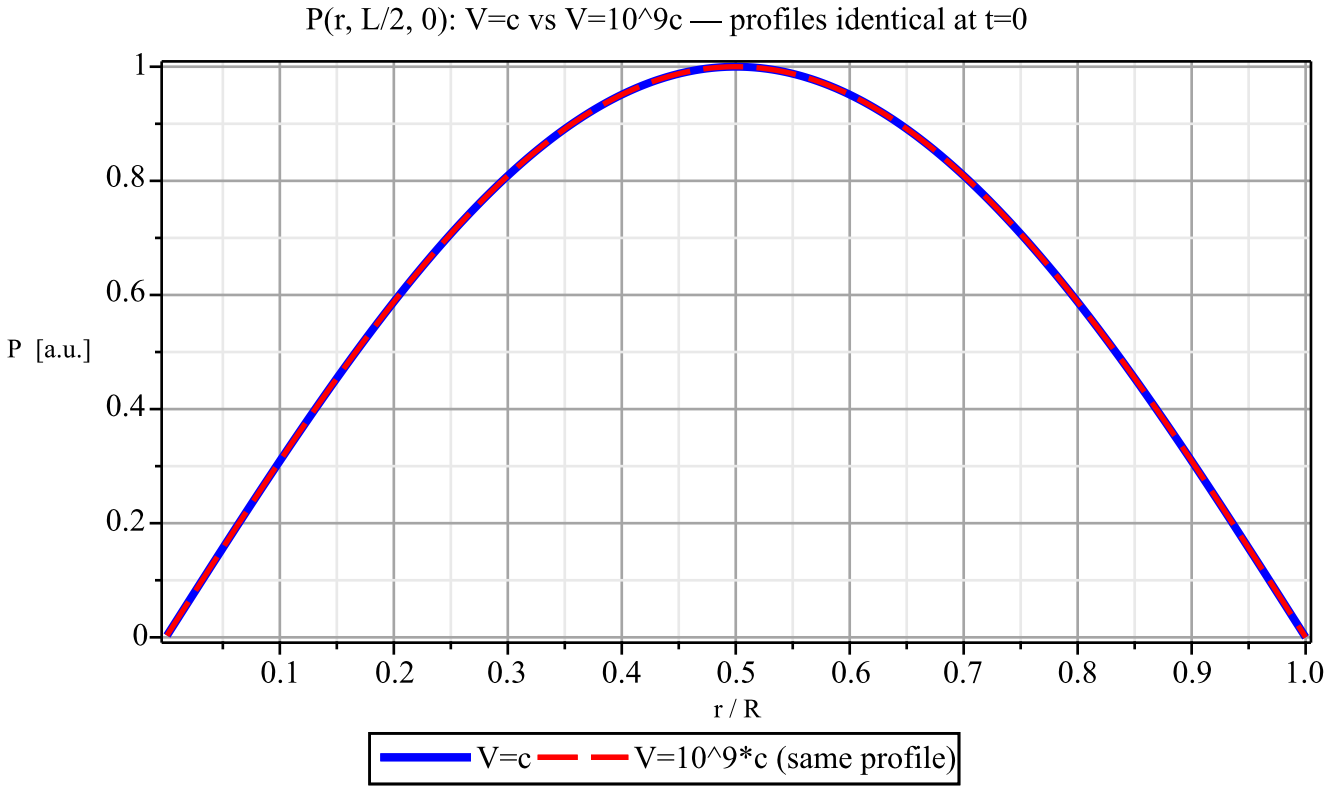
```
=== d'Alembert BC: P(R,z,0,n,m,V) should be 0 ===
P_dA(R, 0, 0, 1, 1, V1) = 3.238e-15
P_dA(R, 0, 0, 2, 1, V1) = -6.477e-15
P_dA(R, 0, 0, 3, 1, V1) = 9.715e-15

=== d'Alembert Frequencies omega_n = n*Pi*V/R ===
n       V=c [rad/s]              V=10^9c [rad/s]
1       2.223638e-14             2.223638e-05
2       4.447275e-14             4.447275e-05
3       6.670913e-14             6.670913e-05
4       8.894550e-14             8.894550e-05
5       1.111819e-13             1.111819e-04
```

Error, (in plots:-pointplot3d) invalid color specification: z

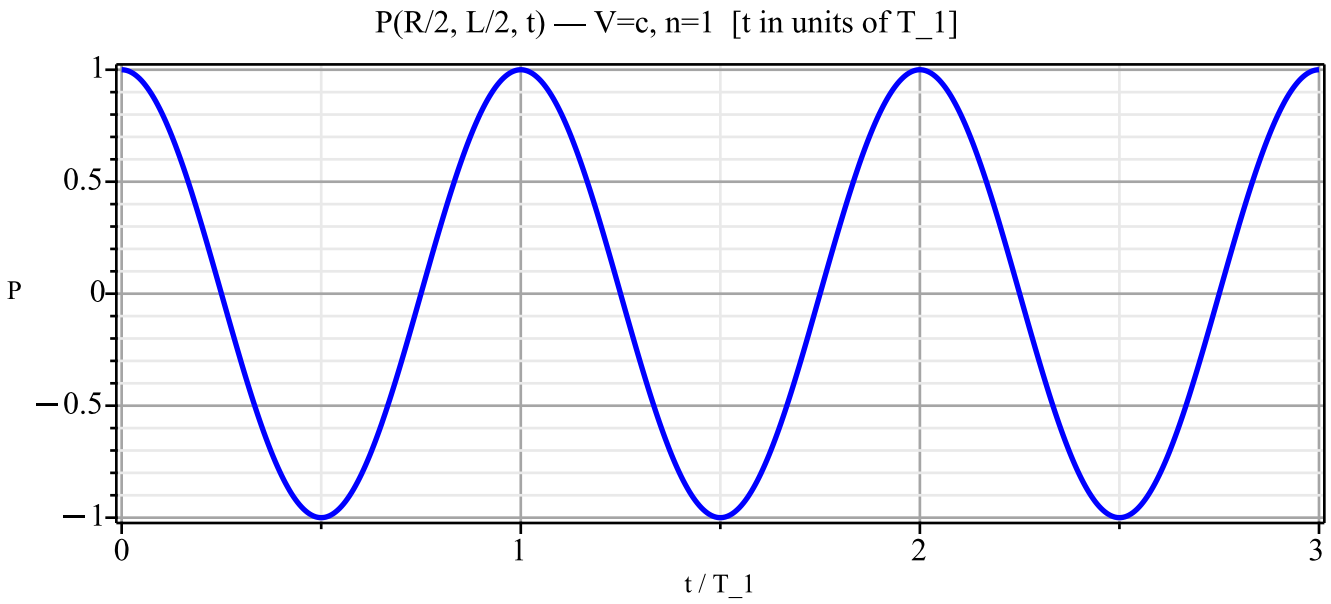
p6

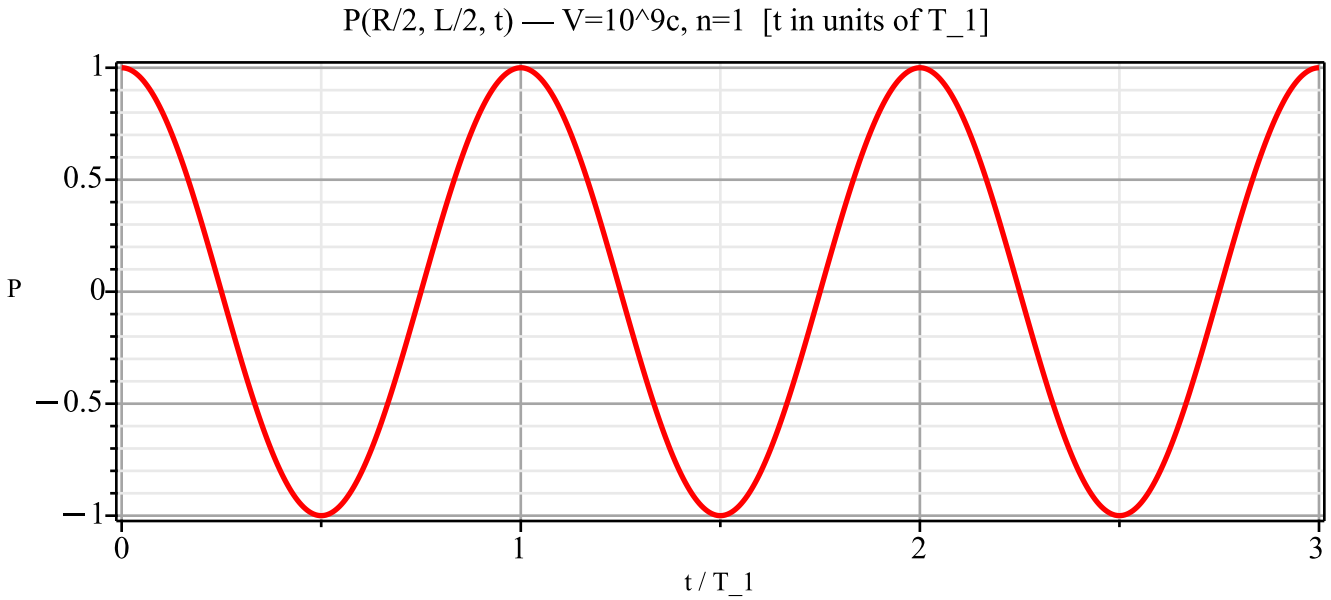





```
=== d'Alembert Periods ===
V=c       : T_1 = 2.8256e+14 s
```

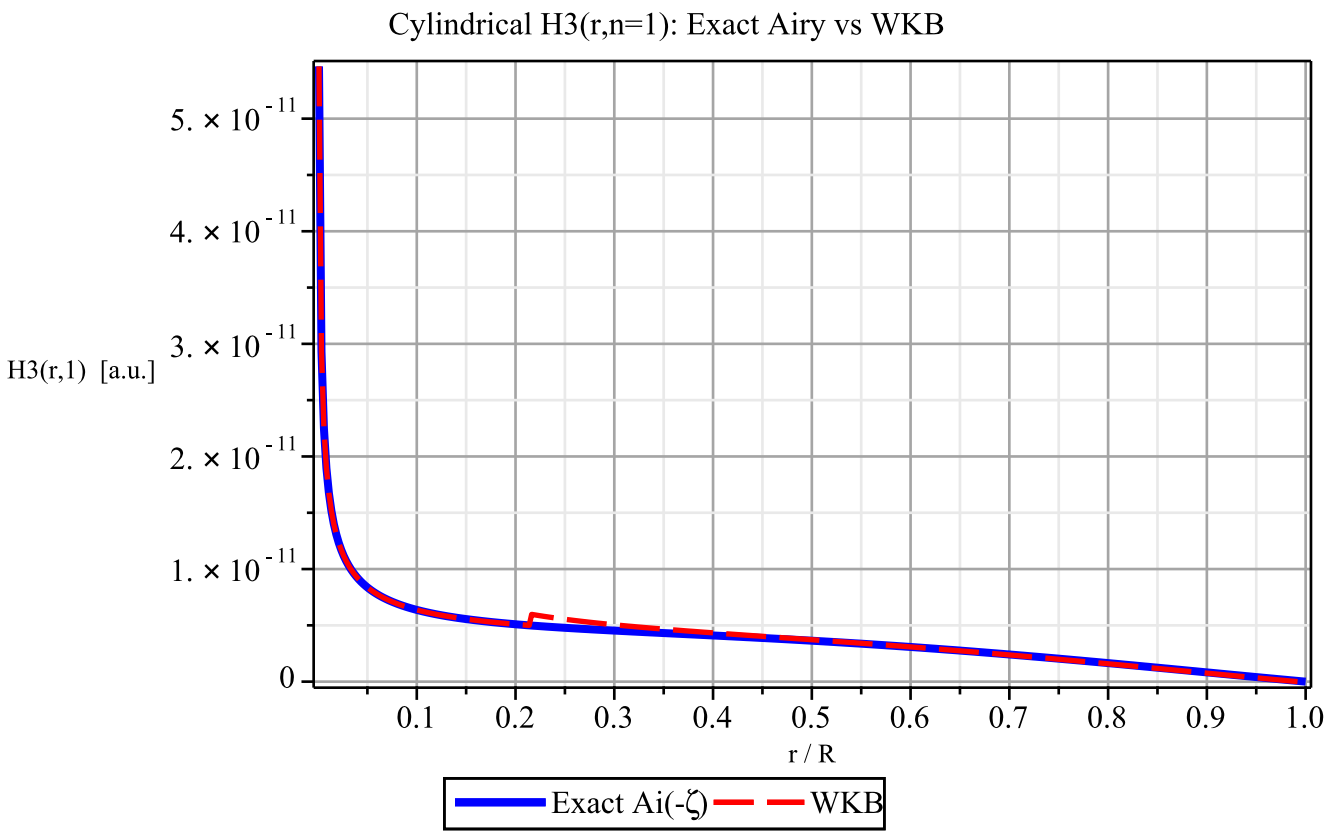
V=10^9*c : T_1 = 2.8256e+05 s

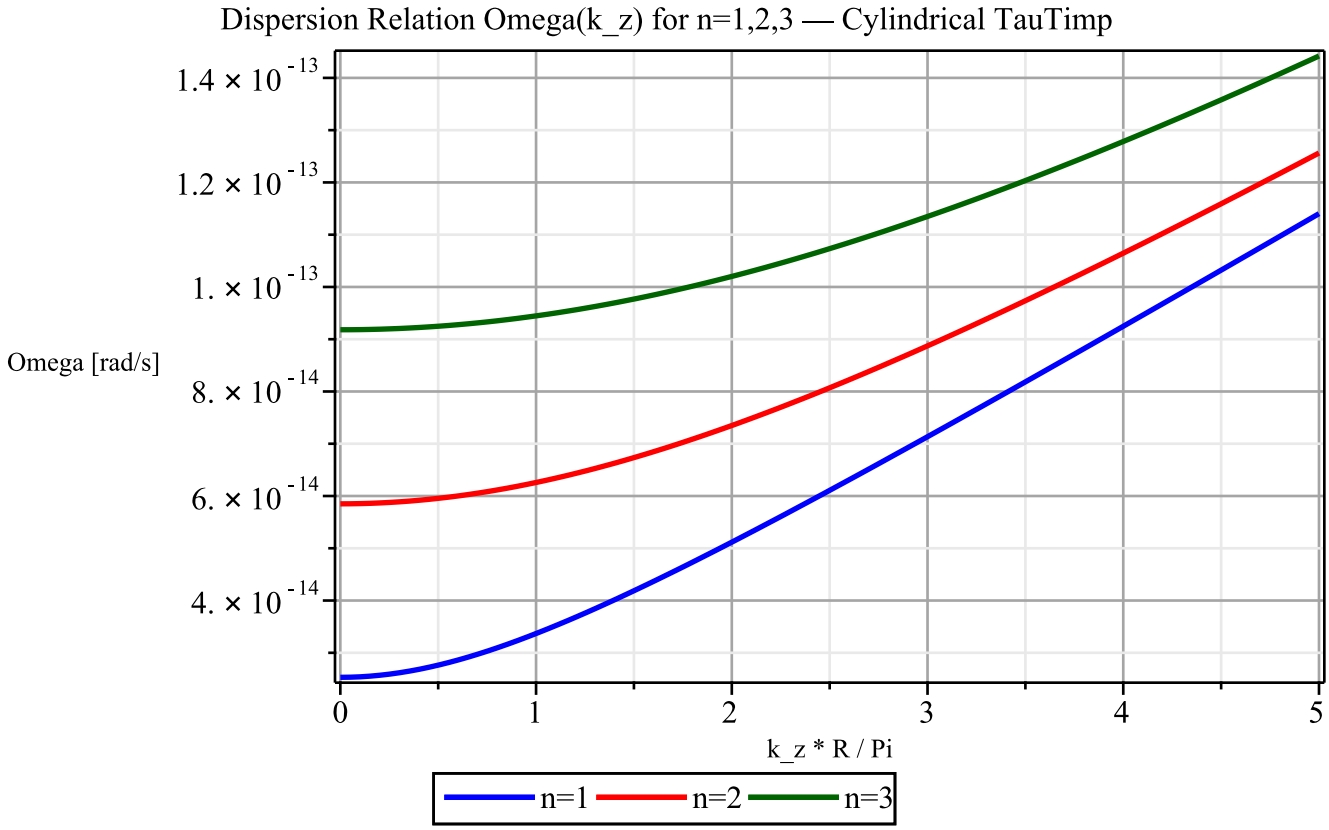




Error, (in plots:-pointplot3d) invalid color specification: z

p10



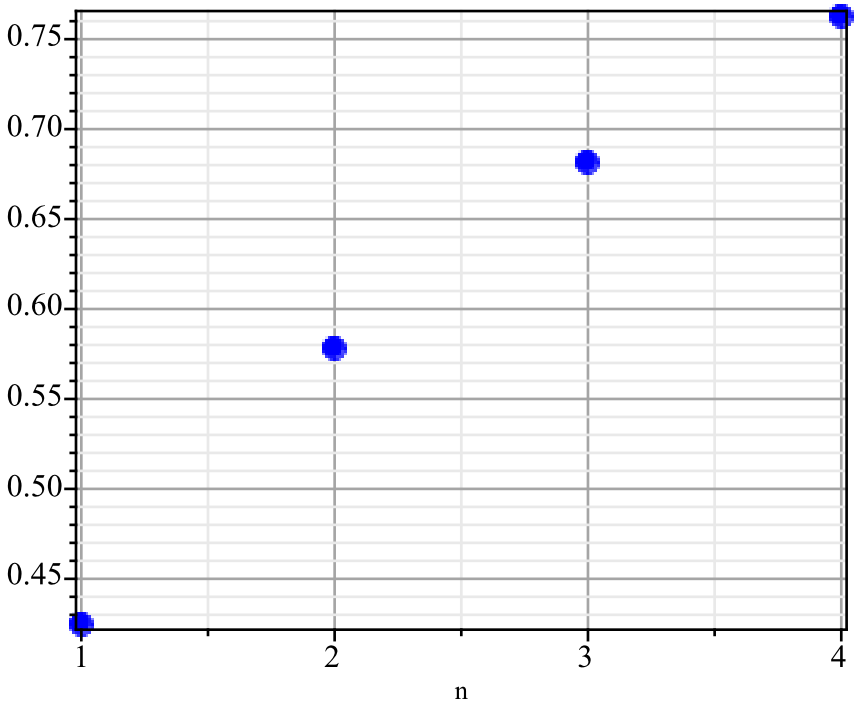



```
=== Normalization Integrals I_n = integral_0^{a_n} [Ai(-rho)]^2 drho
```

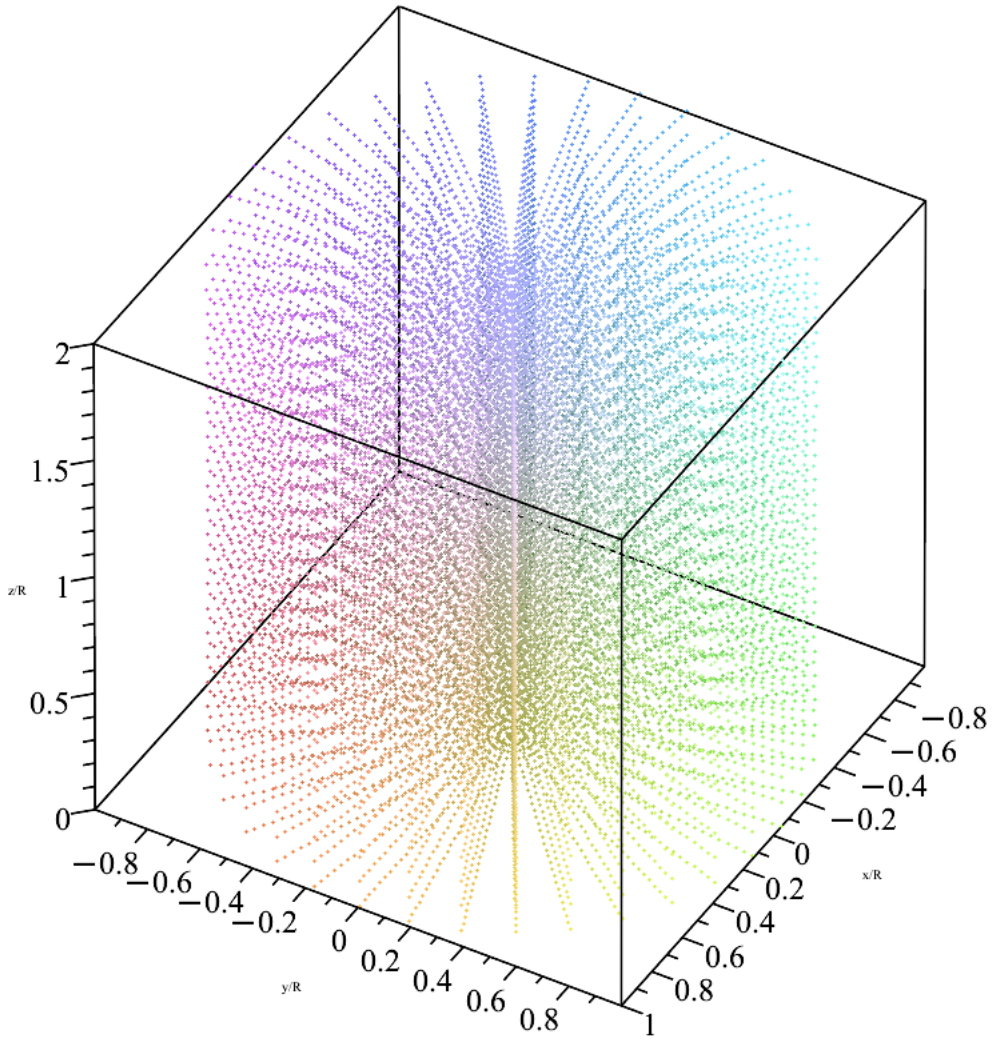
```
===
n=1: I_1=0.424709,   N_1=7.6937e+21 m
n=2: I_2=0.578000,   N_2=5.9887e+21 m
n=3: I_3=0.681591,   N_3=5.2293e+21 m
n=4: I_4=0.762662,   N_4=4.7597e+21 m
```

Normalization integrals I_n vs mode index n

I_n = integral_0^{a_n} [Ai(-rho)]^2 d_rho



H(r,theta,z,0): Cylindrical field with azimuthal modulation




```
================================================================================
========================
  GENERALIZED CYLINDRICAL FRAMEWORK SUMMARY — TauTimp
================================================================================
========================
  R = 4.2355e+22 m,    L_z = 8.4710e+22 m,    c = 2.997925e+08 m/s
--------------------------------------------------------------------------------
------------------------
  n_r      m_z      Method         V [m/s]      Omega [rad/s]            T
[s]
--------------------------------------------------------------------------------
------------------------
  1        0        Airy           c            2.530526e-14
```

```
2.4830e+14
  2       0       Airy          c           5.850223e-14
1.0740e+14
  1       1       Airy          c           2.764001e-14
2.2732e+14
  2       1       Airy          c           5.954935e-14
1.0551e+14
  1       0       d'Alem.       c           2.223638e-14
2.8256e+14
  1       0       d'Alem.       10^9*c      2.223638e-05
2.8256e+05
  2       0       d'Alem.       c           4.447275e-14
1.4128e+14
  2       0       d'Alem.       10^9*c      4.447275e-05
1.4128e+05
================================================================================
======================

=== TauTimp Cylindrical Transcript Complete ===
Plots generated: p1..p14 (14 figures)
Files: TauTimp_CylindricalSymmetry.md + TauTimp_CylindricalSymmetry.
mpl
```